# CS520 Final Project – Midpoint Check-In Guidelines with Examples

## Deadline: October 27, 2025, at 23:59 EST via Gradescope.

This is a group submission. This means that each group only needs to submit their solution once. You will work with students within your group, but not with students from other groups. Multiple groups' submissions may not be created jointly.

Request for extension by contacting TAs at least 24 hours in advance (unless it's a last-minute emergency and you cannot). Medical conditions, religious or funerary events, university-related events (conference visit, athletic event, field trip, or performance), or extenuating non-academic reasons (military obligation, family illness, jury duty, automobile collision) that need extension will be accommodated with written documentation.

### Purpose

The midpoint check-in is designed to ensure that your team is making steady and meaningful progress toward the successful completion of your final project. It is an opportunity to present your progress, demonstrate working code, discuss challenges, and receive feedback from the instructor. This meeting will also help confirm that your project scope and timeline remain realistic.

### What to Prepare

Your team should be ready to present the following during the midpoint check-in:

- Current Progress – Summary of work completed since the proposal phase, including updated diagrams or documents, like architecture diagrams and class diagrams.
- Updated Timeline – A revised Gantt chart or task list showing completed tasks, current work, and projected completion dates.
- Working Demo – A demonstration of at least one or two partially implemented core features (can be CLI, UI, or API).
- Code Repository – A link to your Git repository with visible commit history showing contributions from all team members.
- Challenges & Risks – Any technical or logistical blockers encountered, and strategies to overcome them.
- Next Steps – A plan for remaining work and milestones to reach before final delivery.

> ***Example: At midpoint, a team building a study group app might show their login and course selection screens working, a partially functioning matchmaking feature, and updated architecture***

*diagrams. The GitHub repository would show steady commits over several weeks from all team members.*

---

## Code Deliverable Requirements

- Version Control Evidence – Provide a GitHub/GitLab/Bitbucket link with consistent commits from all members.
- Core Functionality Implementation – At least one or two key features partially implemented and running.
- Runnable Demo – Code should compile/run without major errors, producing functional output.
- Project Structure & Documentation – Organized folders/files and a README with setup/run instructions.
- Testing (Encouraged) – At least minimal unit or integration tests for implemented features.

*Example: For the matchmaking feature, the team might have implemented the database queries and backend API endpoint, with the front-end showing sample matched profiles. The README contains steps to run the backend locally using Node.js and MongoDB.*

---

## Grading Criteria (15 Points Total)

| Criterion | Points | Notes |
|---|---|---|
| **Repository & Commit History** | 3 | Git repo link provided, regular commits, not one bulk commit |
| **Core Feature Implementation** | 4 | At least 1–2 main features partially working and demonstrable |
| **Runnable Demo** | 3 | Code compiles/runs without major errors, functional output |
| **Updated Timeline** | 2 | Reflects actual progress, includes realistic remaining schedule |
| **Challenges & Risk Mitigation** | 2 | Identifies blockers and proposes solutions |
| **Presentation/Discussion Quality** | 1 | Clear and prepared during the meeting |

*Example: A team earns full points for 'Repository & Commit History' by showing a public GitHub repo with commits every few days from*

*all members, including meaningful commit messages like 'Implement login API' and 'Fix bug in course selection form'.*

---

## Collaboration and Contribution Policy

Each member should be able to clearly explain their own contributions during the check-in. The Git commit history and a short written contribution summary from each member will be reviewed. Points for the midpoint check-in will be individually adjusted if a member's participation is below expectations. Non-participation or minimal contribution may result in significant point deductions, up to the full loss of milestone credit.

## Midpoint Discussion Checklist

- Project Summary – Brief restatement of objective & scope.
- Progress Overview – Key milestones achieved since proposal.
- Prototype/Demo – Show working features or design artifacts.
- Updated Timeline – Gantt chart or task list with new dates.
- Challenges/Risks & Mitigation Plans (if any) – List current blockers and risk factors.
- Next Steps – Action plan for remainder of the term.
- Code Review – Repository link, commit history, runnable code.
- Questions/Feedback Needed – Specific areas where you want input.

*Example: In 'Progress Overview', the team shows that they completed user authentication, course selection, and database schema setup, while noting that the matching algorithm is halfway done.*

---

## Submission Requirements

Prior to the meeting, submit a Midpoint Progress Document via GradeScope containing:
- Brief project summary
- Progress since the proposal
- Updated timeline (Gantt chart or equivalent)
- Current issues and mitigation plans (if any)
- Next steps
- Link to your repository with instructions to run the code